CSED226 데이터분석 입문 practice2 report

컴퓨터공학과 20240940 장해영

1. Introduction

해당과제는 csv파일로 주어진 많은 데이터를 효율적으로 처리하는 방법을 학습하는 과제로, 데이터로드, 이상치 제거, 그룹화, 데이터 재구조화 등을 pandas를 이용하여 해결하는 과제이다. 주어진 data는 nycflights에 대한 dataset으로 수많은 비행편들에 대한 비행시간, 출발지, 도착지, 항공편 정보 등 비행정보를 포함하고 있다. 이 data들을 이용하여 6가지의 data handling을 하는 것이 이번 과제이다.

2. Outputs and explanations for each tasks & difference with reference code

A. Task1: data loading, type conversion, and initial cleaning

```
Shape: (32735, 18)
Memory (MB): 7.23605
year                int64
month               int64
day                 int64
dep_time            int64
dep_delay           int64
arr_time            int64
arr_delay           int64
carrier          category
tailnum            object
flight              int64
origin           category
dest               object
air_time            int64
distance            int64
hour                int64
minute              int64
dep_time_hour     float64
arr_time_hour     float64
dtype: object
```

- 옆 사진은 task1을 수행한 결과이다. csv파일을 로드한 뒤, dep_time과 arr_time을 주어진 형식으로 변환한 값을 dep_time_hour, arr_time_hour에 각각 저장하여 출력하였다. 또한, carrier과 origin의 경우 데이터 타입을 category로 지정하여 출력하였다. 마지막으로 결측치는 제거한 데이터를 df_clean에 저장하여 다음 task에서 이용할 수 있도록 하였다.
- reference code에서는 dropna를 이용하여 결측치를 제거하였지만, 나의 코드에서는 loc을 이용하여 결측치가 아닌 것만을 살리는 방법으로 코드를 구성하였다.

B. Task2: outlier detection and robust cleaning

```
Outliers removed: 2817
delay_category
short     24956
medium     4962
long          0
Name: count, dtype: int64
```

- 옆 사진은 task2를 수행한 결과이다. 주어진 범위 내에 있는 값들만 필터링 하는 과정으로, 이상치를 제거하는 과제이다. 이렇게 필터링을 완료한 데이터들을 df_filtered에 저장하여 다음 과제에서 이 데이터를 사용할 수 있도록 하였다.
- 해당 task에서는 reference code와의 차이는 없었다. 결과값도 동일한 것을 확인할 수 있다.

C. Task3: advanced grouping and custom aggregation

	origin	carrier	mean_arr_delay	prop_long_delay	median_air_time	p95_distance
0	LGA	FL	4.805147	0.0	109.0	762.0
1	LGA	F9	1.682540	0.0	227.0	1620.0
2	LGA	YV	1.638298	0.0	77.0	544.0
3	EWR	EV	1.637268	0.0	91.0	1092.0
4	JFK	MQ	0.947287	0.0	68.0	1005.0

- origin과 carrier 두 그룹으로 묶은 뒤 mean_arr_delay를 내림차순으로 상위 5개의 데이터를 출력하는 task이다. 이 때 출력하는 데이터는 origin, carrier, mean_arr_delay, prop_long_delay, median_air_time, p95_distance이다.

- reference code는 agg()를 이용한 뒤 round(2)를 추가하여 소수점 두 번째 자리까지만 나타내도록 하였지만, 나의 코드는 이런 과정을 거치지 않아 소수점 여섯 번째 자리까지 나타내어졌다. 이러한 차이로 2번째와 3번째 행이 바뀐 결과로 보인다.

- groupby()를 사용하는 과정에서 버전의 차이로 observed와 관련된 오류가 계속 발생함을 확인하고 gemini를 이용하였다. observed = True를 추가하면 오류가 발생하지 않는다는 것을 확인하고 추가하여 오류를 해결하였다. 이는 아래 과제의 groupby에서도 마찬가지로 이용되었다.

D. Task4: reshaping and pivot analysis

```
Shape: (36, 4) Corr: -0.04
```

- melt를 이용하여 reshape을 하는 과제이다. 그 후 mean_dep_delay와 month의 상관계수를 계산하여 소수점 둘째 자리까지 나타내었다.

- reference code의 출력결과 중 shape이 (36, 3)인 것을 확인하였다. 이는 margin의 포함여부로 인해 발생한 차이로 보인다. 나의 코드는 margin = True로 하여 ALL이 포함되었지만, reference code에서는 포함하지 않아 합계를 제거했기 때문에 1이 더 작은 것으로 추측할 수 있다. 과제파일을 다시 읽어보았을 때 ALL을 제거하는 방향이 더 올바른 방향인 것 같다.

E. Task5: window functions and ranking

```
month
1     AS
2     EV
3     EV
4     B6
5     FL
6     YV
7     YV
8     FL
9     FL
10    F9
11    MQ
12    AS
Name: carrier, dtype: category
Categories (16, object): ['9E', 'AA', 'AS', 'B6', ..., 'US', 'VX', 'WN', 'YV']
carrier
FL    5.034166
F9    0.853108
EV    0.488422
Name: cumulative_avg, dtype: float64
```

- 해당 task에서는 mean_arr_delay를 기준으로 carriers를 dense rank로 매긴 뒤 지연시간이 가장 긴 상위 12개의 carrier를 출력하고, 각 carrier 별 월평균 arr_delay을 계산하여 지연시간이 가장 긴 상위 3개의 carrier를 평균값과 함께 출력한다.

- 위의 과제에서와 마찬가지로, reference code에서는 소수점 아래 둘째자리까지만 반올림하여 나타내었지만, 나의 코드에서는 round를 이용하지 않아 소수점 아래 여섯 번째 자리까지 나타내어졌다. 높은 정확성을 요구하는 과제였다면, 내가 했던 방식으로 소수점아래 여섯번째 자리까지 나타내는 것이 좋았겠지만, 해당과제에서는 매우 높은 정확성을 요구하지는 않았기에, 적당히 조정하여 소수점 아래 둘째자리까지만 나타내어도 괜찮았을 것 같다.

F. Task6: complex filtering and derived insights

dest	mean_score	count
SFO	-0.067691	1159

- distance가 1000보다 크고, air_time이 90번째 백분위 수보다 크다는 조건을 우선으로 만족하고, 주어진 식으로 count를 계산하여 이 count가 1000이상인 조건 총 3개의 조건을 모두 만족하는 carrier에 대해 count에 대해 하위 5개의 carrier를 출력하는 것이 task6이다. 원래는 5개의 carrier를 출력해야 하지만, 조건을 만족하는 carrier가 1개밖에 존재하지 않아 하나만 출력된 것을 확인할 수 있다. 이는 reference code에서도 하나만 출력된 것을 통해 확인할 수 있었다.

- 해당 task에서는 reference code와의 차이는 없었다. 결과값도 소수점 아래 자릿수를 제외하고, 동일한 것을 확인할 수 있다.

3. Result and Lessons

데이터 전처리부터 집계, 변환, 윈도우 함수까지 pandas의 고급 기능을 실제 데이터셋에 적용하면서, 데이터를 필터링하는 방법에 대해 학습하였다. 또한 reference code와 비교하며 더 효율적이고 재사용성 높은 코드 작성 습관을 익힐 수 있었다. 나의 코드와 reference code의 가장 큰 차이점이었던 "소수점 아래 자릿수를 얼마로 지정할 것인가"를 다시 한 번 생각해볼 수 있는 계기가 되었다.